Enterprise Employee Analytics

Architecture, Setup, Access Control and Data Warehousing Logic

Greenfield Mini-Project | Subhaprada, Shravani, Keerthi

October 2026

Live application: https://enterprise-employee-analyticsfinal-fezpmjni2cruxyezl9vfva.streamlit.app/

1. Overview

Enterprise Employee Analytics is an end-to-end employee data platform. HR users and managers sign in to a Streamlit web application and enter operational data (employees, projects, assignments, performance reviews). The data is stored in a normalized OLTP MySQL database, moved by an ETL pipeline and stored procedures into a star-schema data warehouse (OLAP) that keeps employee history using SCD Type 2, and analysed on an interactive dashboard that uses SQL window functions. Employees sign in to see their own profile. Both databases are hosted on Aiven MySQL and the application is deployed on Streamlit Community Cloud.

- **Live application:** https://enterprise-employee-analyticsfinal-fezpmjni2cruxyezl9vfva.streamlit.app/
- **Source code:** https://github.com/Shravani-A-Kulkarni/Enterprise-Employee-Analytics
- **Technologies:** Python 3.11+, pandas, Faker, MySQL 8 (stored procedures, CTEs, window functions), mysql-connector-python, Streamlit, Plotly, python-dotenv, pytest, Git and GitHub, MySQL Workbench, Aiven MySQL, Streamlit Community Cloud.

Business problem. Employee information sits in operational tables, so it is hard to compare performance across years, to see who may leave, and to notice when people are over-allocated on projects. The warehouse and dashboard give HR and managers one trusted place for reporting and analytics, and role-based access keeps salary and personal data limited to the people who need it.

2. System architecture

2.1 Components

Layer	Technology and files	Responsibility
Data source	IBM HR Analytics dataset, `python/data_synthesizer.py`	Scale 1,470 real rows to 100,000 synthetic employees and create history records
OLTP database	MySQL database `enterprise_employee_analytics`	Normalized operational storage for employees, projects, assignments, reviews and user logins (`AppUser`)
ETL	`python/etl_pipeline.py`, `02_olap_dml.sql`, `06_app_support.sql`	Extract from OLTP, clean and transform, load the warehouse
OLAP warehouse	MySQL database `enterprise_employee_analytics_olap`	Star schema for analytics with SCD Type 2 history
Python backend	`app/db_manager.py`, `entities.py`, `managers.py`, `auth.py`	Object-oriented data access layer with validation, transactions, password hashing and login lookup
Router and access control	`app/streamlit_app.py`, `PAGE_ACCESS` in `auth.py`	Shows only login and sign-up when signed out; builds the sidebar from the user's role when signed in

Layer	Technology and files	Responsibility
Web application	`app/login.py`, `signup.py`, `app/pages/`	Sign-in, data entry forms, analytics dashboard, employee profile and directory
Hosting	Aiven MySQL, Streamlit Community Cloud, GitHub	Cloud databases, public application, version control

2.2 Architecture diagram

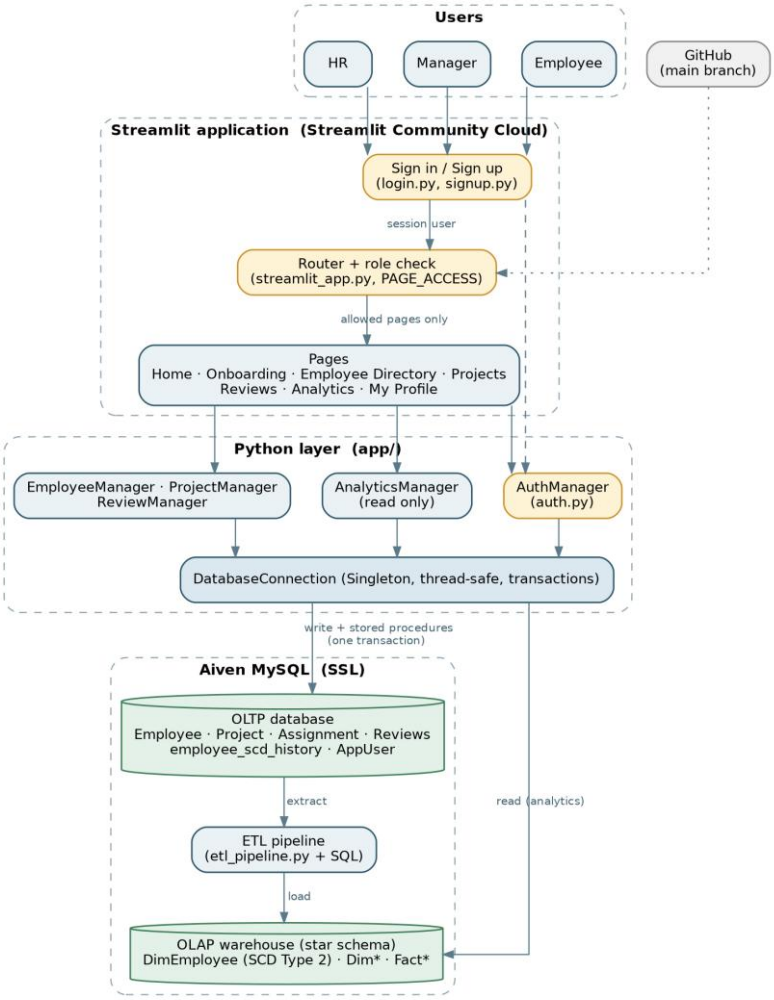


Figure 1. Architecture including the login and role layer (amber boxes)

2.3 Data flow

```
User (HR / Manager / Employee)
  -> Login or Sign-up  (login.py / signup.py)
     -> Router + role check  (streamlit_app.py, PAGE_ACCESS)
        -> Streamlit pages  (app/pages/)
           -> Python layer  (auth.py, entities.py, managers.py, db_manager.py)
              -> OLTP MySQL database
                 -> ETL pipeline
                    -> OLAP data warehouse (star schema)
                       -> Analytics SQL (CTEs, LAG, DENSE_RANK, RANK)
                          -> Analytics dashboard (Plotly charts)

GitHub     -> Streamlit Community Cloud (runs the application)
Aiven Cloud -> hosts MySQL OLTP + OLAP (SSL connection)
```

2.4 Write path, read path and login path

Write path (saving data). The user submits a Streamlit form on a page their role can open. The entity class validates the input. A manager class opens one database transaction, writes to the OLTP tables, and calls a warehouse stored procedure. If every step succeeds the transaction commits; if any step fails everything is rolled back, so the OLTP database and the warehouse never disagree.

Read path (analytics). The ETL pipeline loads OLTP data into the warehouse. The dashboard calls `AnalyticsManager`, which runs SQL against the star schema only, and Plotly draws the charts. Query results are cached for a few minutes and the cache is cleared after every write.

Login path. The login form calls `AuthManager.authenticate`. On success the user (`user_id`, `email`, `role`, `display_name`, `employee_number`) is stored in `st.session_state["user"]`. The router then builds the sidebar from `PAGE_ACCESS[role]`, so pages the role cannot open do not exist for that session. Log out clears the session state.

2.5 Python architecture

Class (file)	Role
`DatabaseConnection` (`db_manager.py`)	Singleton: one shared, thread-safe MySQL connection per class. Methods: `query`, `query_df`, `execute`, `transaction` (commit or rollback), `test_connection`. Reads settings from environment variables, a `.env` file or Streamlit Secrets.
`AuthManager` (`auth.py`)	Login and accounts on the OLTP database: `authenticate`, `create_user`, `sign_up`, `email_exists`, `linkable_employee_number`, `find_employee_for_user`. Also holds `hash_password`, `verify_password`, `validate_signup` and `PAGE_ACCESS`.
`EmployeeManager` (`managers.py`)	OLTP: employee CRUD, lookups, read-only `list_directory` (basic columns only), SCD Type 2 update, version history, delete only when the employee has no reviews or assignments
`ProjectManager`	OLTP: create projects, assign employees (duplicates rejected), list teams, check total allocation
`ReviewManager`	OLTP: save and list performance reviews, then sync them to the warehouse
`AnalyticsManager`	OLAP: company KPIs and the four analyses (read only)
`Employee`, `Project`, `Review` (`entities.py`)	Entities with private attributes and validating properties
`ui_helpers.py`	Shared page helpers: `page_setup`, `current_user`, `require_roles`, cached dropdown lookups, `clear_dashboard_cache`

Design points. All managers inherit from `DatabaseConnection` and each is its own Singleton; OLTP managers and `AuthManager` use the OLTP database, while `AnalyticsManager` uses the warehouse. Entity classes validate every input before it reaches the database: person names accept letters only (spaces, hyphens and apostrophes are allowed inside a name), the email must look like name@domain.tld, ages must be 18 to 70, ratings 1 to 5, review dates cannot be later than 2027-12-31 (the end of `DimDate`), and project names and project roles reject special characters. Invalid input raises `ValueError`, and database problems are raised as a single `DatabaseError` type that the pages display. All values are passed as query parameters (never concatenated into SQL), and a re-entrant lock protects the shared connection because Streamlit serves each user in its own thread.

2.6 Hosting

GitHub holds the code. Streamlit Community Cloud runs `app/streamlit_app.py` from the main branch. Aiven MySQL hosts both databases, and the application connects over SSL using the Aiven

CA certificate. Passwords are never stored in the code: they live in a git-ignored `.env` file locally and in Streamlit Secrets in the cloud.

3. Authentication and access control

3.1 Roles and page access

Page access comes from the `PAGE_ACCESS` dictionary in `app/auth.py`. To change who can open a page, edit that dictionary.

Page	HR	Manager	Employee
Home	Yes	Yes	Yes (welcome message only)
Onboarding (add, update, delete employees)	Yes	-	-
Employee Directory (read-only)	-	Yes	-
Projects	-	Yes	-
Reviews	Yes	Yes	-
Analytics	Yes	Yes	-
My Profile	-	-	Yes

3.2 Sign-in flow

1. The user opens the app. While nobody is signed in, only the **Sign in** and **Sign up** pages exist.
2. `AuthManager.authenticate` looks up the email in `AppUser` (active accounts only) and verifies the password against the stored hash.
3. On success the user is saved in the session and the app reruns; the router now shows only the pages allowed for that role.
4. **Log out** in the sidebar clears the session and returns to the sign-in page.

3.3 Self sign-up and linking to an employee record

Anyone can create an account from the sign-up page, but it always creates an **Employee** login. HR and Manager accounts are created by an administrator (section 3.6).

- **Checks:** full name of 2 to 150 characters, a valid email, a password of at least 8 characters with a letter and a number, and a matching confirmation.
- **Duplicates:** an email that already has an account is rejected, including two people signing up at the same moment.
- **Automatic link:** if an `Employee` row has exactly the same email, and no other login is already linked to it, the new login is linked to that employee. Nothing is guessed from names.
- **No match:** the login stays unlinked and no employee record is created. My Profile then shows a clear message asking the person to contact HR.

Identity on My Profile. Which employee a signed-in user may see is decided on the server from the `AppUser` row (found by the `user_id` stored at sign-in). It never comes from the URL, a form field or anything the browser could change, so an employee can only ever open their own record. If the login is not linked but exactly one employee has the same email (and no other login claims it), the page falls back to that match and shows a note that the link is not saved yet.

3.4 Password security

Topic	How it works
Storage	Never in plain text. Stored as `pbkdf2_sha256$200000$<salt>$<hash>`: HMAC-SHA256, 200,000 iterations, a 16-byte random salt per user.
Verification	`hmac.compare_digest` compares in constant time. For an unknown email a dummy hash is verified, so the response time does not reveal whether the email exists.
SQL safety	All values are bound as query parameters; no SQL is built from user text.
Second line of defence	The router hides pages a role cannot open, and the Onboarding and Employee Directory pages also call `require_roles(...)` before showing anything.
Least data	The manager Employee Directory shows only ID, name, email, department, role, level and Active/Left status. Salary, age, gender, marital status and ratings are not selected by its query.

3.5 Demo accounts

`sql/10_app_users.sql` creates four demo logins. The password for all of them is `Demo@123`. The script is safe to re-run.

Email	Role	Sees
`hr@company.com`	HR	Home, Onboarding, Reviews, Analytics
`manager@company.com`	Manager	Home, Employee Directory, Projects, Reviews, Analytics
`employee1@company.com`	Employee	Home, My Profile (employee #1)
`employee2@company.com`	Employee	Home, My Profile (employee #2)

> **Before a public demo:** change these passwords or delete the accounts, and set `SHOW_DEMO_LOGINS = "0"` in `.env` or Streamlit Secrets to hide the demo-accounts box on the login page.

3.6 Managing user accounts

HR and Manager accounts. They cannot be created from the sign-up page. An administrator creates them from the `app` folder, with the database settings in `.env`:

```
cd app
python -c "from auth import AuthManager; AuthManager().create_user('jane@company.com',
'StrongPass1', 'Manager', 'Jane Doe')"
```

Use `HR`, `Manager` or `Employee` as the role. The password is hashed before it is saved.

Linking an existing login. If an Employee login exists before its employee record, it stays unlinked. After HR onboards the employee with the same email, My Profile finds the record on the next visit. To save the link permanently, use the optional script `sql/11_link_app_user_to_employee.sql`. It is manual: step 1 lists safe candidates, step 2 links one account (edit the email inside it first), step 3 checks the result. The application never runs it.

4. Application pages

Page	Roles	What it does
Home	All	Employees see a welcome message. HR and managers see OLTP and OLAP connection status, company KPIs (current employees, attrition rate, average income, historical SCD versions, reviews, projects) and a guide listing only the pages their role can open.
Onboarding	HR	Three tabs. Onboard employee: form with name, age, gender, marital status, department, role, level, education, salary, travel, distance, overtime, working years and email. Update employee (SCD Type 2): change department, role, level or salary, see the warehouse version history, and delete an employee that has no reviews or assignments. Directory: search by name or number with a department filter.
Employee Directory	Manager	Read-only search by employee ID, name or email with a department filter (up to 200 rows, newest first), and a one-click way to copy an employee ID for assigning to projects.
Projects	Manager	Create project (name, department, status, dates), Assign employee (search the employee, pick the project, role and allocation percentage; a warning appears when total allocation exceeds 100%), and Projects and teams (status filter and team list).
Reviews	HR, Manager	Submit review (1 to 5 rating, review date up to 2027-12-31, optional comments up to 500 characters) and Review history for one employee or the latest reviews of everyone.
Analytics	HR, Manager	Sidebar filters (department, top N per department, year) and four tabs: year-over-year performance, top performers, attrition risk, project bottlenecks. A Refresh button clears the cache.
My Profile	Employee	Four tabs for the signed-in employee only: My details, My projects, My reviews and Career history (every SCD Type 2 version of department, role, level and salary).

5. Data sourcing and synthesis

The source is the public IBM HR Analytics Employee Attrition and Performance dataset (1,470 rows, a single snapshot in time). The class `EmployeeDataSynthesizer` in `python/data_synthesizer.py` uses pandas and Faker to create 100,000 employees: each employee starts from a randomly chosen real row, then receives a new employee number, a fake first and last name and an email address of the form first.last<id>@company.com.

Because a snapshot has no history, the script also creates historical records for 10% of employees (10,000). For each of them it writes an older version valid from 2023-01-01 to 2024-12-31 with a lower salary (85% to 95% of the current salary), sometimes a lower job level and sometimes a different department, and a current version valid from 2025-01-01. This gives 20,000 history rows that make SCD Type 2 meaningful. The CSV files are loaded into a staging table in MySQL and then into the OLTP tables.

6. OLTP database design

The operational database is normalized: repeated text is moved into lookup tables and referenced through foreign keys.

Table	Key columns	Role
`Department`	department_id (PK), department_name	Lookup, 3 rows

Table	Key columns	Role
JobRole	job_role_id (PK), job_role_name	Lookup, 9 rows
EducationField	education_field_id (PK), education_field_name	Lookup, 6 rows
Employee	employee_id (PK), employee_number (unique), names, email (unique, nullable), age, income, foreign keys to department, job role and education field, survey columns, attrition	Main entity, 100,000 rows
Project	project_id (PK), name, department, start and end date, status	Projects
Assignment	assignment_id (PK), employee_id, project_id, role, allocation_percentage	Many-to-many link between employees and projects
Reviews	review_id (PK), employee_id, review_date, review_period, rating, comments	Performance reviews
employee_scd_history	history_id (PK), employee_number, department, role, level, income, start and end date, is_current	Version history of employees
AppUser	user_id (PK), email (unique), password_hash, role (HR / Manager / Employee), display_name, employee_number (nullable link), is_active, created_at	Logins and roles

The ER diagram was designed in MySQL Workbench and is stored in the `diagrams` folder of the repository. It was drawn before `AppUser` and `Employee.email` were added, so use the table above for those two.

7. Data warehousing logic

7.1 Star schema

The warehouse (database `enterprise_employee_analytics_olap`) is a star schema: fact tables hold measurements, dimension tables describe them.

Table	Type	Grain and content
FactEmployee	Fact	One row per employee version: income, satisfaction scores, attrition and other measures, plus the keys of its dimensions
FactAssignment	Fact	One row per assignment: allocation percentage
FactReview	Fact	One row per review: rating
DimEmployee	Dimension (SCD Type 2)	One row per employee version, with start date, end date and is_current
DimDepartment, DimJobRole, DimEducationField	Dimensions	Small lookup dimensions
DimProject	Dimension	Project attributes
DimDate	Dimension	One row per calendar day (year, quarter, month, day name), 2023 to 2027

The verified load produced: DimDepartment 3, DimJobRole 9, DimEducationField 6, DimProject 30, DimDate 1,096, DimEmployee 110,000 (100,000 current and 10,000 historical), FactEmployee 100,000, FactAssignment 26,508 and FactReview 104,945. DimDate was later extended to the end of 2027 so that new reviews can be dated in 2026 and 2027.

7.2 Surrogate keys and natural keys

Every dimension has a surrogate key (for example `employee_key`, `department_key`) generated by the warehouse. `employee_number` is the natural key from the source system. Because one employee can have several versions, `employee_number` is not unique in `DimEmployee`; each version has its own `employee_key`, and fact rows point to the version that was valid for them. Surrogate keys also keep the warehouse independent of source-system keys.

7.3 SCD Type 2 logic

Slowly Changing Dimension Type 2 preserves history. When a tracked attribute changes, the old row is not overwritten: it is closed with an end date and `is_current = 0`, and a new row with a new surrogate key is inserted with `is_current = 1`.

employee_key	department	job_level	income	start date	end date	is_current
110004	Sales	2	6000	2025-01-01	2026-10-03	0
110005	Research & Development	3	7500	2026-10-04	NULL	1

Illustrative example of one employee with two versions.

How a change is applied. When HR saves a change of department, job role, job level or monthly income, `EmployeeManager.update_employee` runs one transaction that updates the Employee table; closes the current row in `employee_scd_history` (or creates the baseline row first if the employee had no history) and inserts the new current row; then calls the stored procedure `sp_scd2_update_employee`. The procedure follows these rules:

1. Find the current row: `employee_number = X AND is_current = 1`.
2. If no tracked attribute really changed, do nothing (no new version).
3. Close the old row: `effective_end_date` = new start date minus 1 day, `is_current = 0`. If the new date would not be after the old start date, the start is moved to old start plus 1 day so date ranges stay valid.
4. Insert the new version with a new surrogate key: changed values, `effective_start_date` = new start date, `effective_end_date = NULL`, `is_current = 1`.
5. Insert a `FactEmployee` row for the new version so current-state reports still include the employee.

The history table is updated in the same transaction, so re-running the ETL reproduces the same versions and never loses a change made in the application.

Why it matters. SCD Type 2 lets reports answer questions as of a date. A review is attached to the employee version that was valid on the review date (procedure `sp_sync_review`), so year-over-year results by department stay correct even after the employee moves to another department. Employees can also see their own full history on the My Profile page.

7.4 ETL logic

The pipeline in `python/etl_pipeline.py` (with the SQL load script `02_olap_dml.sql`) follows Extract, Transform, Load.

- **Extract.** Read employees, history, projects, assignments and reviews from the OLTP database into pandas.
- **Transform.** Remove duplicate records and records with missing mandatory identifiers (the verified run removed 0 of each from 100,000 employees). Compare incoming employees with `DimEmployee` and classify them as new, changed or unchanged; only new or changed employees produce new rows. Look up surrogate keys and date keys for the fact tables.
- **Load.** Fill the dimensions first (department, job role, education field, project, date, employee), then the fact tables, because fact rows need the dimension keys.

- **Re-runnable.** Simple dimensions use unique constraints so they load without duplicates, and the fact tables are cleared and reloaded.

7.5 Advanced SQL used

- **Stored procedures.** Three reporting procedures (`sp_attrition_by_department`, `sp_salary_by_job_role`, `sp_performance_summary`) and five application procedures: `sp_scd2_update_employee`, `sp_sync_new_employee`, `sp_sync_project`, `sp_sync_assignment` and `sp_sync_review`. The application procedures do not control transactions themselves; the Python manager opens the transaction so OLTP and warehouse changes commit or roll back together.
- **CTEs (WITH).** Used to build the analysis queries in readable steps.
- **Window functions.** `LAG()` for year-over-year change, `DENSE_RANK()` and `RANK()` for rankings.

7.6 Analytics logic

Year-over-year performance. A CTE averages review ratings per year and department (using the department at the time of the review). `LAG(avg_rating) OVER (PARTITION BY department_name ORDER BY review_year)` returns the previous year's value for the same department; the difference is the year-over-year change.

Top performers. Employees are ranked inside each department with `DENSE_RANK() OVER (PARTITION BY department_name ORDER BY avg_rating DESC, review_count DESC, performance_rating DESC, monthly_income DESC)`. Employees with equal values share a rank and there are no gaps; the dashboard shows ranks up to a chosen N.

Attrition risk. Each current employee who has not left receives a score from 0 to 8, one point per condition:

Factor	Rule
Overtime	`overtime = Yes`
Low job satisfaction	`job_satisfaction <= 2`
Poor work-life balance	`work_life_balance <= 2`
Low environment satisfaction	`environment_satisfaction <= 2`
No promotion for years	`years_since_last_promotion >= 5`
Long commute	`distance_from_home >= 20`
Underpaid	`monthly_income` below 80% of the department average
Short tenure	`years_at_company <= 2`

Bands: **High** is 5 or more, **Medium** is 3 or 4, **Low** is 0 to 2. The dashboard also shows attrition rate by department and by job role, and the list of highest-risk employees.

Project bottlenecks. A first CTE totals each employee's allocation across all projects; an employee above 100% is over-allocated. A second CTE counts, for each project, its team size, average allocation and over-allocated members. `RANK()` orders projects by the number of over-allocated members, so rank 1 is the biggest bottleneck. The dashboard also lists the most over-allocated employees.

8. Setup instructions

8.1 Prerequisites

Python 3.11 or newer, MySQL 8 (MySQL Workbench is helpful), and Git.

8.2 Local setup

```
git clone https://github.com/Shravani-A-Kulkarni/Enterprise-Employee-Analytics.git
cd Enterprise-Employee-Analytics
python -m venv .venv
.venv\Scripts\activate            # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
copy .env.example .env            # macOS/Linux: cp .env.example .env
# then open .env and fill in your MySQL values
```

Then prepare the databases in MySQL Workbench, in this order:

1. Run `sql/schema.sql` to create the OLTP database and load the CSV data (edit the CSV file paths inside it for your machine). It already includes the `email` column.
2. Run `sql/01_olap_ddl.sql`, then `sql/02_olap_dml.sql` to create and load the warehouse.
3. Run `sql/07_seed_demo_data.sql` once. It adds projects, assignments and reviews and corrects the overtime column. For speed, first run `CREATE INDEX idx_staging_empno ON employee_staging (EmployeeNumber);`
4. Run `sql/06_app_support.sql` to create the application stored procedures and extend DimDate.
5. Run `sql/10_app_users.sql` to create the `AppUser` login table and the four demo accounts (employees 1 and 2 must exist).
6. Load the new rows into the warehouse: `python python/etl_pipeline.py`
7. Verify the database layer: `python tests/smoke_test.py` (it must end with ALL CHECKS PASSED).
8. Optionally run the mocked unit tests: `pip install pytest`, then `python -m pytest tests -q`.
9. Start the application: `streamlit run app/streamlit_app.py` (it opens at http://localhost:8501) and sign in with a demo account.

> **Upgrading an existing database?** If your `Employee` table has no `email` column, run `sql/09_add_employee_email.sql` once before using the new pages. In MySQL Workbench, raise Edit > Preferences > SQL Editor > DBMS connection read timeout to 3600 seconds before running the long scripts.

8.3 Configuration

Settings are read from environment variables, the `.env` file, or Streamlit Secrets. The `.env` file is never committed.

Variable	Meaning
`MYSQL_HOST`, `MYSQL_PORT`	Database server address and port
`MYSQL_USER`, `MYSQL_PASSWORD`	Database credentials
`OLTP_DB`	`enterprise_employee_analytics`
`OLAP_DB`	`enterprise_employee_analytics_olap`
`MYSQL_SSL_CA`	Path to the CA certificate for hosted MySQL (for example `ca.pem`)
`SHOW_DEMO_LOGINS`	Optional. `1` (default) shows the demo-accounts box on the login page; `0` hides it

9. Cloud deployment

1. **Create the hosted database.** Create a free Aiven MySQL service and note its host, port, user and password; download the CA certificate as `ca.pem`.

2. **Move the data.** Export both local databases with MySQL Workbench Data Export (leave out the `employee_staging` table and the stored procedures), connect Workbench to Aiven with SSL, and import the file.
3. **Fix names and procedures.** The hosted server runs on Linux, where table names are case-sensitive, while the Windows export contains lowercase names. Run `sql/08_aiven_setup.sql` once on Aiven: it restores the table names (Employee, DimEmployee and so on) and creates the five application stored procedures.
4. **Add the login features.** On the OLTP database run `sql/09_add_employee_email.sql` (only if the imported Employee table has no email column) and `sql/10_app_users.sql` (creates `AppUser` with the correct capitalisation and the demo accounts).
5. **Test against Aiven.** Put the Aiven values and `MYSQL_SSL_CA=ca.pem` in `.env`, run `python tests/smoke_test.py`, then start the app locally and sign in.
6. **Deploy.** On Streamlit Community Cloud create an app from the GitHub repository (branch main, main file `app/streamlit_app.py`) and paste the settings into Secrets in TOML format, with `MYSQL_SSL_CA` set to `ca.pem`. Add `SHOW_DEMO_LOGINS = "0"` for a public site.
7. **Verify.** Open the public URL: sign in as HR and check that the home page shows both databases Connected, then confirm that each role sees only its own pages.

Security notes. The connection uses SSL with the Aiven certificate. `.gitignore` currently lists `ca.pem`, so make sure the file reaches the deployed app (for example `git add -f ca.pem`; the CA certificate is a public file and safe to commit). Passwords exist only in `.env` (git-ignored) and Streamlit Secrets. Change the demo passwords on a public site. Git work was done on feature branches merged into main through pull requests. The Aiven free plan has 1 GB of storage (the project used about 158 MB before the login table was added) and the service sleeps when idle.

10. Testing and results

10.1 Pytest suite (mocked, no database needed)

209 test functions in 12 files check the application logic without touching a database. Run them with `python -m pytest tests -q` (install pytest first).

File	What it covers
`test_auth.py`	Password hashing and verification, role-to-page matrix, authenticate, create_user, sign-up validation, duplicate emails and the sign-up race, exact-email linking, find_employee_for_user
`test_login.py`, `test_signup_coverage.py`	Login and sign-up pages: success, invalid input, database error
`test_streamlit_app.py`	Router: login shown when signed out, allowed pages per role
`test_my_profile.py`	Linked account sees its own record; unlinked account is blocked
`test_employee_directory.py`	Search, department filter, hidden sensitive columns, access guard
`test_onboarding_coverage.py`, `test_projects_coverage.py`	Onboarding and Projects pages
`test_managers.py`, `test_entities.py`	Manager classes and entity validation
`test_db_manager.py`	Singleton, connection settings, SSL, transactions, error handling
`test_ui_helpers.py`	Shared page helpers: current_user, require_roles and others

10.2 Database smoke test

`tests/smoke_test.py` runs 41 checks against the real databases and ends with ALL CHECKS PASSED. It covers the Singleton and both connections, entity validation, onboarding and warehouse loading, SCD Type 2 (new version, exactly one current row, closed old row, no change means no new version), rollback after a failed update, project creation, assignment (including duplicate rejection), review submission (including invalid ratings), KPIs, year-over-year, top-N, attrition, risk and bottleneck queries. It creates a few test rows and may leave them in the database, so do not run it casually against production data.

10.3 End-to-end testing

The deployed application was also tested end to end: sign-in as each role, onboarding, employee directory, SCD Type 2 update, project creation, assignment, review submission and history, all four analytics tabs, My Profile, and the connections to both databases on Aiven.

11. Troubleshooting

Symptom	Likely cause and fix
Home page shows a database as Down	The free Aiven service is asleep: wait a minute and reload. Otherwise check the values in `.env` or Streamlit Secrets.
`Table 'AppUser' doesn't exist`	Run `sql/10_app_users.sql` on the OLTP database.
`Unknown column 'email'`, or onboarding fails	Run `sql/09_add_employee_email.sql` once.
Invalid email or password for a demo account	Demo accounts exist only after `10_app_users.sql`, and the employee demos need employees 1 and 2 in the Employee table. The password is `Demo@123`.
My Profile says the account is not linked	HR has not created an employee record with the same email as the login, or another login already claims that employee (section 3.6).
A review is rejected because of its date	Review dates must be on or before 2027-12-31 (the end of `DimDate`).
Table-not-found errors on Aiven	Table names are case-sensitive on Linux: run `sql/08_aiven_setup.sql`.
SSL connection error	Check that `MYSQL_SSL_CA` points to `ca.pem` and that the file is present next to the app.
Signed out after refreshing the browser	Sign-in lives in the Streamlit session, so a full page refresh asks you to sign in again.

12. Limitations and future scope

Limitations.

- The data is synthetic, so the insights demonstrate the system rather than real company facts.
- The attrition score is a transparent rule-based model, not machine learning.
- Authentication is intentionally simple: no password reset, no email verification, no account lockout or rate limiting, no multi-factor sign-in, and no screen for HR to create or deactivate logins (this is done with SQL or the Python snippet in section 3.6).
- A signed-in session ends when the browser page is refreshed.
- The ETL is run manually, and the free cloud database sleeps when idle.
- The demo accounts share a known password and are meant for demonstration only.

Future scope. Predictive attrition models, password reset and email verification, an admin screen for managing users, login rate limiting, scheduled automatic ETL, more dashboards and KPIs, alerts for high-risk employees, project capacity planning, and CI/CD automation that runs the pytest suite on every pull request.

13. Team and contributions

Part	Owner
Data synthesis and OLTP database design	Subhaprada
Data warehouse design and ETL	Shravani
Python OOP backend, Streamlit application, analytics dashboard and deployment	Keerthi

Appendix A. SQL script reference

Script	Database	Purpose
`schema.sql`	OLTP	Create the OLTP schema (includes email) and load the CSV data
`01_olap_ddl.sql`	OLAP	Create the star-schema tables
`02_olap_dml.sql`	OLAP	Load dimensions and facts
`03_stored_procedures.sql`	OLAP	Three reporting procedures
`04_cte_queries.sql`, `05_window_functions.sql`	OLAP	Example CTE and window-function queries
`06_app_support.sql`	OLAP	Five application procedures and DimDate extension to 2027
`07_seed_demo_data.sql`	OLTP	Demo projects, assignments and reviews; overtime fix. Run once
`08_aiven_setup.sql`	Aiven	Restore table-name capitalisation and create the application procedures
`09_add_employee_email.sql`	OLTP	Add Employee.email (unique, nullable) to an existing database. Run once
`10_app_users.sql`	OLTP	Create AppUser and the four demo accounts. Safe to re-run
`11_link_app_user_to_employee.sql`	OLTP	Optional and manual: link an existing login to the employee with the same email